

Studify

Use-Case Specification: Validate Login Credentials

Version: 1.0

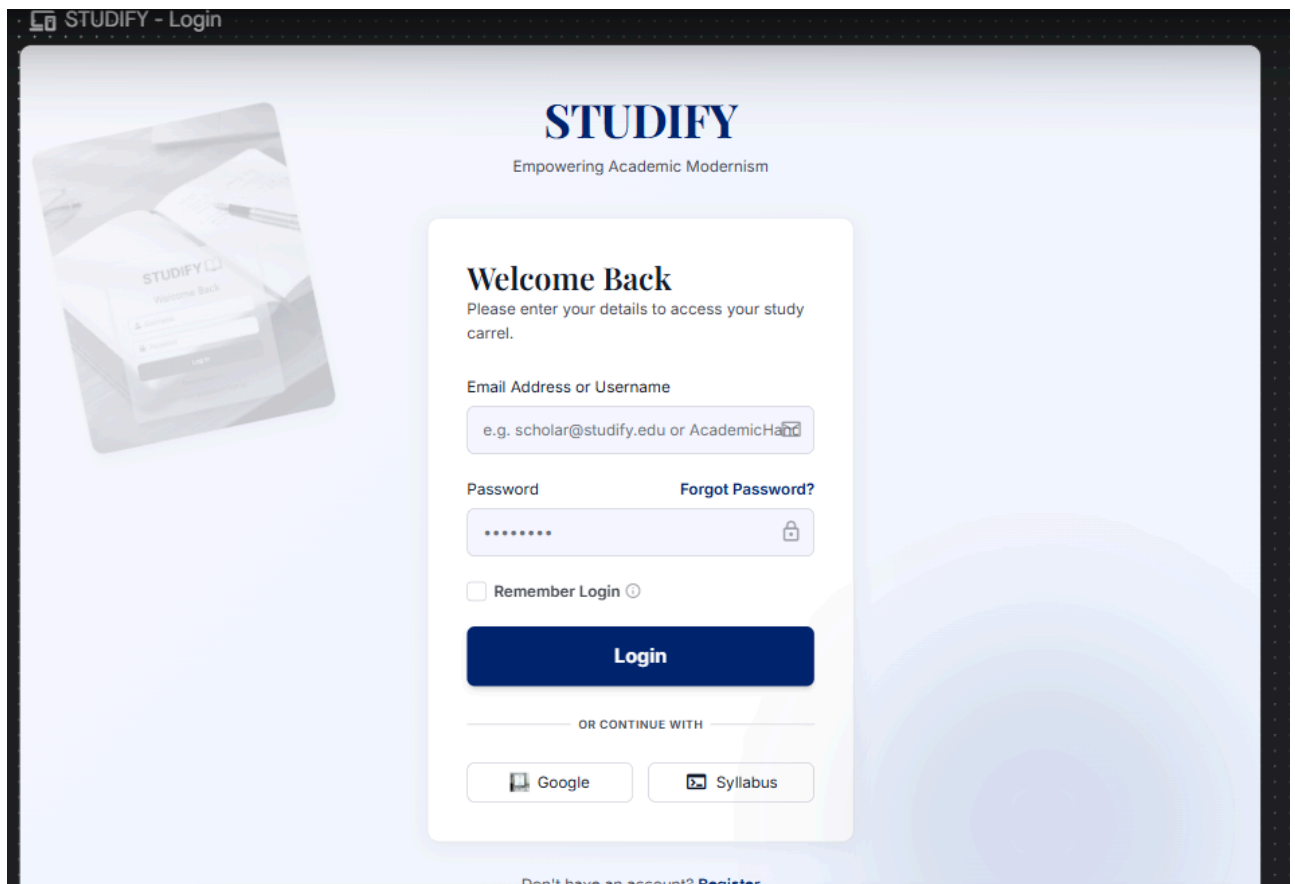
Revision History

Date	Version	Description	Author
23/Jul/26	1.0	Initial version of UC7	Khanh Linh

Table of Contents

- [1. Use-Case Name](#)
 - [1.1 Brief Description](#)
- [2. Flow of Events](#)
 - [2.1 Basic Flow](#)
 - [2.2 Alternative Flows](#)
 - [2.2.1 Account Not Found](#)
 - [2.2.2 Incorrect Password](#)
 - [2.2.3 Account Locked](#)
- [3. Special Requirements](#)
- [4. Preconditions](#)
- [5. Postconditions](#)

- 6. Extension Points



1. Use-Case Name

UC8 — Validate Login Credentials

1.1 Brief Description

This use case describes the system's internal process of verifying that the Email/Username and Password submitted during login match a stored, active account. It is always executed (included) as part of UC2 — Login, and is not directly invoked by an actor; it is a system-level check triggered automatically whenever a login attempt is made.

2. Flow of Events

2.1 Basic Flow

This use case begins automatically when UC2 — Login reaches the credential-checking step, immediately after the Registered User submits the login form.

1. The system receives the submitted Email/Username and Password values from UC2.
2. The system looks up the account record matching the submitted Email/Username.
3. The system checks that the account is active (not locked or suspended; see Alternative Flow 2.2.3).
4. The system hashes the submitted Password using the same algorithm used at registration and compares it against the stored password hash.

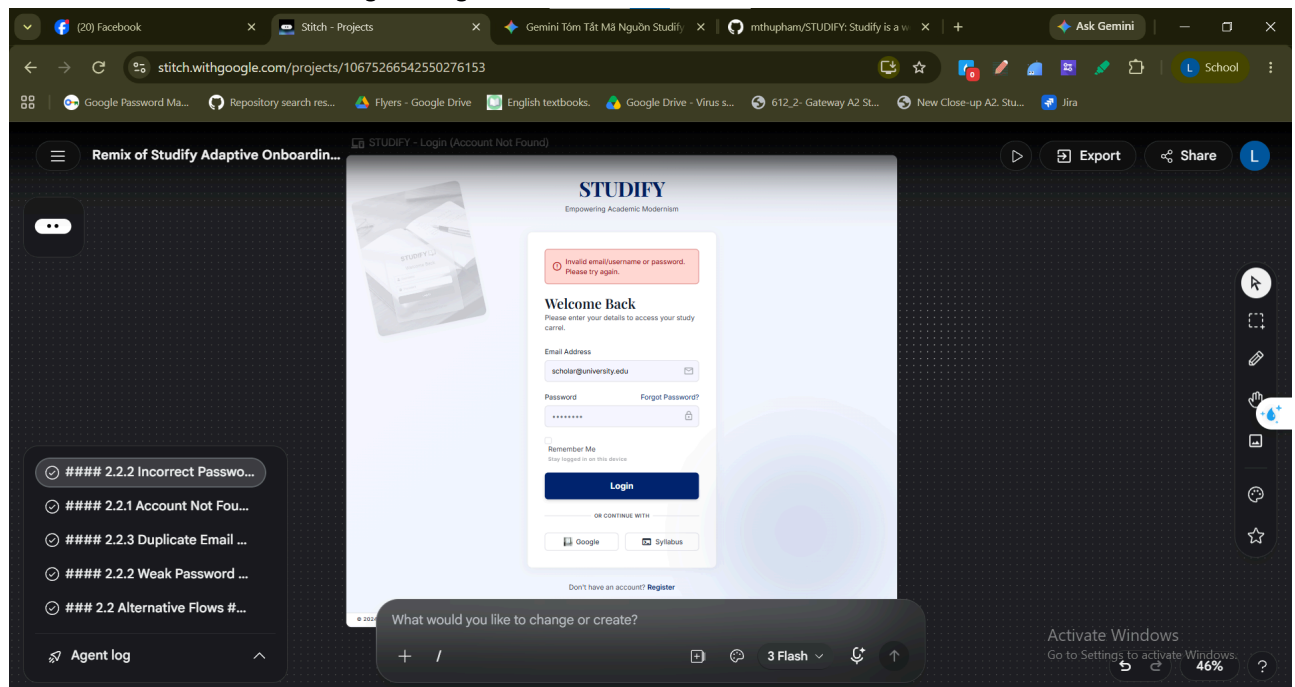
5. If the account exists, is active, and the password hash matches, the system returns a "valid" result to UC2, allowing the login process to proceed.
6. The use case ends successfully.

2.2 Alternative Flows

2.2.1 Account Not Found

At step 2 of the Basic Flow, if no account matches the submitted Email/Username:

1. The system returns a "validation failed" result to UC2, using a generic reason (not revealing whether the email/username itself exists) to avoid account enumeration.
2. UC2 displays the corresponding generic error message to the Registered User (see UC2, Alternative Flow 2.2.1).
3. The system logs the failed attempt for security monitoring purposes.
4. The use case ends without granting access.

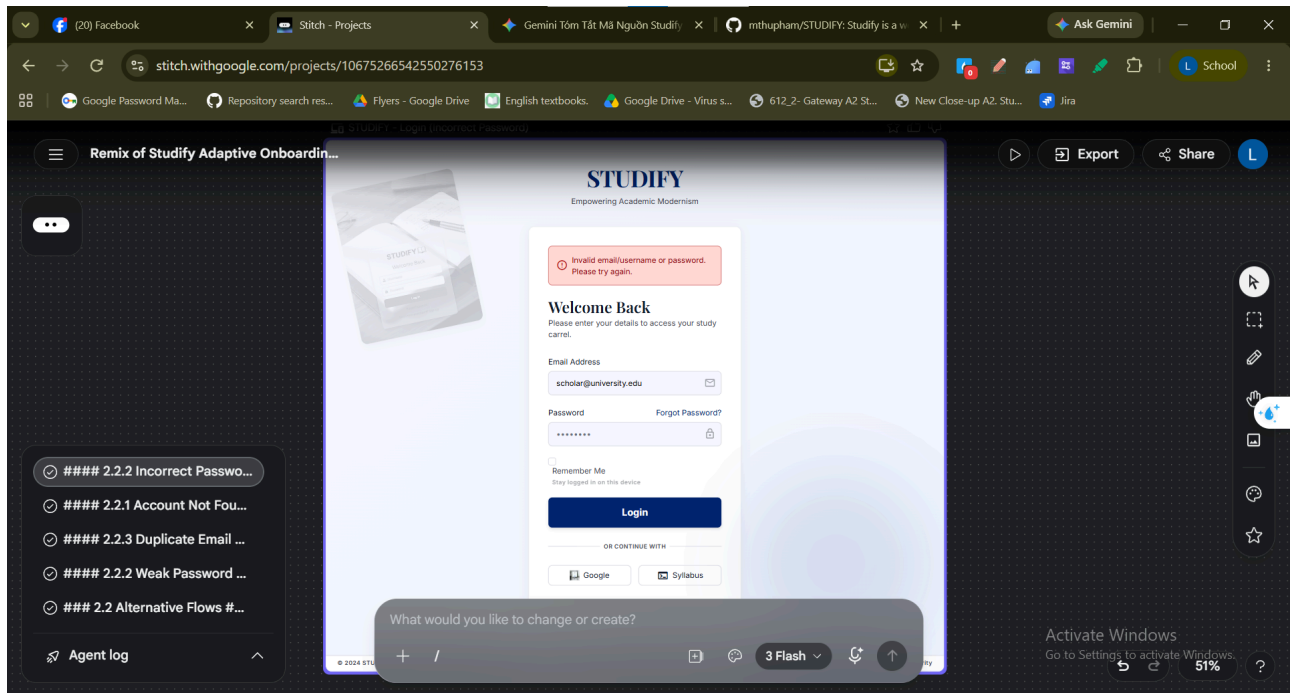


2.2.2 Incorrect Password

At step 4 of the Basic Flow, if the account exists and is active, but the submitted password does not match the stored hash:

1. The system returns a "validation failed" result to UC2, using the same generic reason as in 2.2.1, to avoid revealing that the email/username was correct.
2. The system increments the account's consecutive failed-login counter.
3. UC2 displays the corresponding generic error message to the Registered User (see UC2, Alternative Flow 2.2.1).

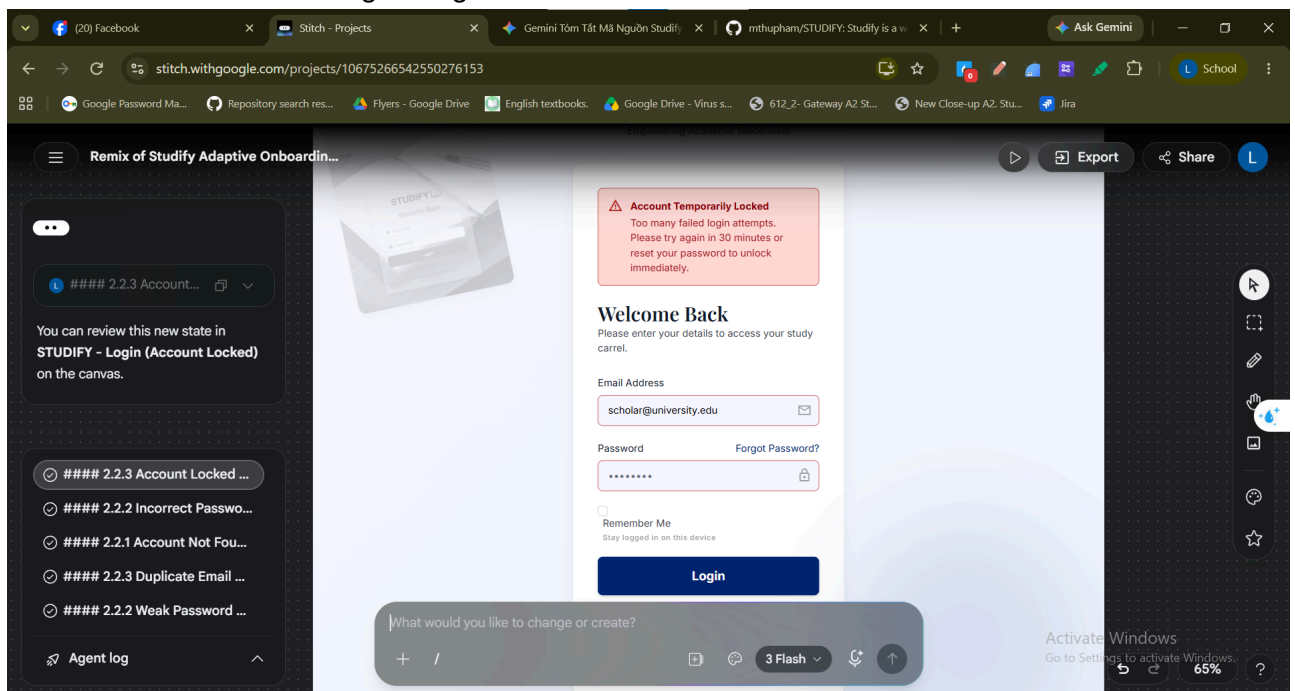
4. The use case ends without granting access.



2.2.3 Account Locked

At step 3 of the Basic Flow, if the account is found to be locked or suspended (e.g., due to too many consecutive failed login attempts, per the Account Lockout Policy):

1. The system returns a "validation failed" result to UC2, indicating the account is temporarily locked.
2. UC2 displays a message to the Registered User informing them of the lockout and, if applicable, how long until they may retry or how to unlock the account (e.g., via password reset).
3. The use case ends without granting access.



3. Special Requirements

3.1 Response Time

Credential validation, including password hash comparison, should complete within 500 milliseconds under normal load to keep the login flow responsive.

3.2 Timing-Attack Resistance

The system should take a similar amount of time to respond whether the account does not exist (2.2.1) or the password is incorrect (2.2.2), to prevent attackers from inferring account existence through response-time differences.

3.3 Failed Attempt Logging

All failed login attempts should be logged with a timestamp and, where feasible, source information (e.g., IP address) to support the Account Lockout Policy and security auditing.

3.4 Password Hash Algorithm

Password comparison must use a secure, industry-standard hashing algorithm (e.g., bcrypt) consistent with the algorithm used during registration (UC7).

4. Preconditions

4.1 Triggered by Login

This use case can only be initiated as part of UC2 — Login; it cannot be triggered independently by any actor.

5. Postconditions

5.1 Validation Result Returned

A clear "valid" or "validation failed" (with an internally logged reason, though not necessarily disclosed to the user) result is returned to UC2, determining whether the login process may proceed.

5.2 Failed Attempt Recorded (on failure)

If validation fails due to incorrect credentials, the account's failed-attempt counter is incremented, potentially leading to a lockout state.

6. Extension Points

6.1 None

This use case is included by UC2 — Login and does not currently define further extension points into other use cases.